

Credit Card Approval Prediction: AI-Powered Risk Assessment Engine

Project Description

Credit Card Approval Prediction leverages Machine Learning classification models to automate credit card application screening, providing financial firms a swift, data-backed assessment framework. The system architecture processes demographic profile fields, history scores, and multi-factor income vectors to estimate the likelihood of defaults. It implements high-accuracy classifiers including Support Vector Machines (SVM), Random Forests, and Logistic Regression to produce reliable credit worthiness predictions.

Built with a clean Python Flask routing core engine backend and exposed locally via customized visual frames, the solution optimizes continuous numerical cleaning routines and scales out raw parameters via robust feature data standardizers to safeguard operational boundaries.

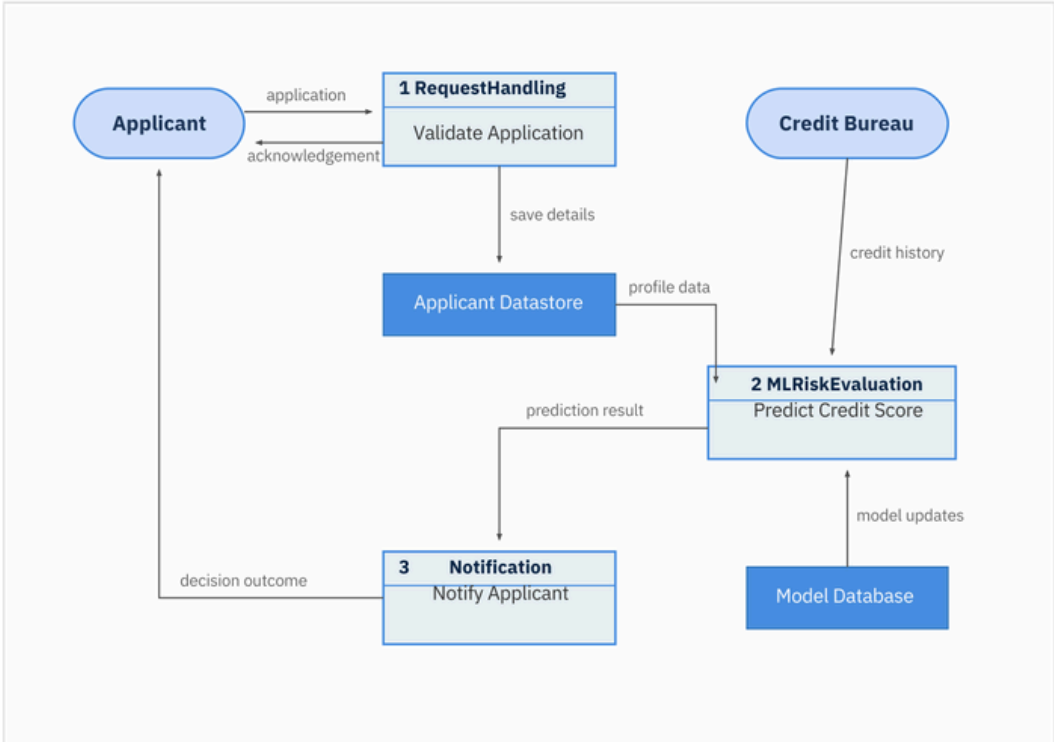
Scenarios

Scenario 1: An underwriting officer receives an application with high annual salary matrices, steady employment tenure histories, and stellar credit ratings. The system applies the pre-trained weights binary parameters to return a verified risk assessment flag showing an instant "Approved" verdict with low-risk vector confidence margins.

Scenario 2: A marginal evaluation scenario contains irregular self-employed billing parameters alongside elevated external liabilities. The engine safely intercepts the records, parses out weight metrics, and returns a balanced classification score flagging the account for detailed secondary tier checks or manual oversight review.

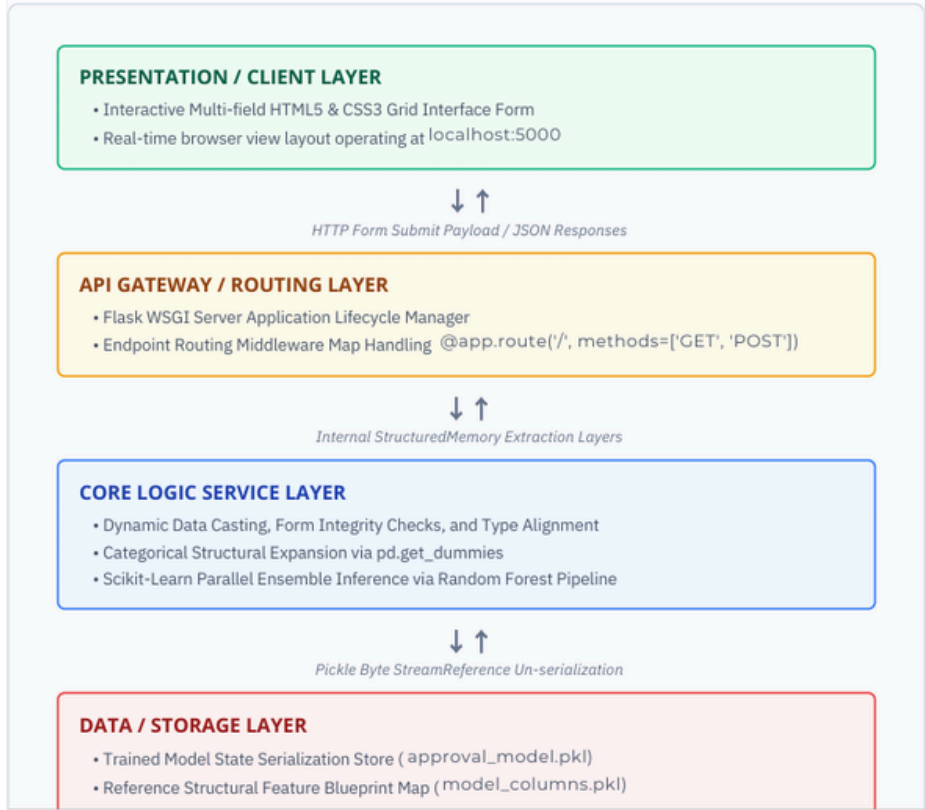
Scenario 3: An incoming request sequence shows negative ledger profiles or critical delinquent flags. The pipeline immediately maps the data matrix across boundary thresholds, executing a definitive classification output indicating "Rejected" status to guarantee system security.

Technical Architecture



SOLUTION ARCHITECTURE DIAGRAM

The visual representation below outlines the layered micro-framework architecture map designed for the predictive system, showing communication loops between presentation, handling components, and structural weight stores.



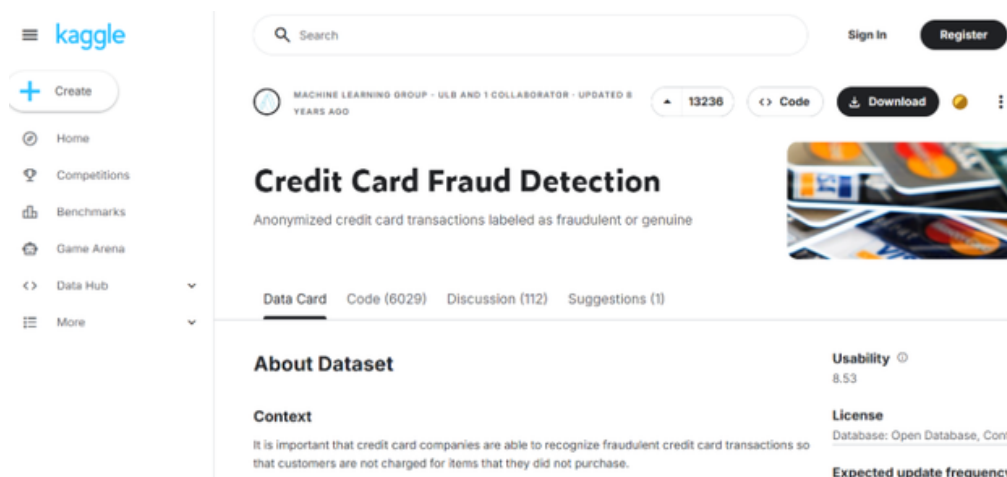
Pre-requisites

- **Flask Web Application Framework:** Component orchestrating routing bindings and request loops.
- **Scikit-Learn Infrastructure:** Library powering data standardizer instances and core classifier arrays.
- **Python Core Environment:** Version 3.10+ execution runtimes containing necessary mathematical libraries.
- **Pandas & NumPy Arrays:** Engines tracking vector matrix transformations and data formatting.

Project Workflow & Implementation Steps

Milestone 1: Model Selection and Architecture

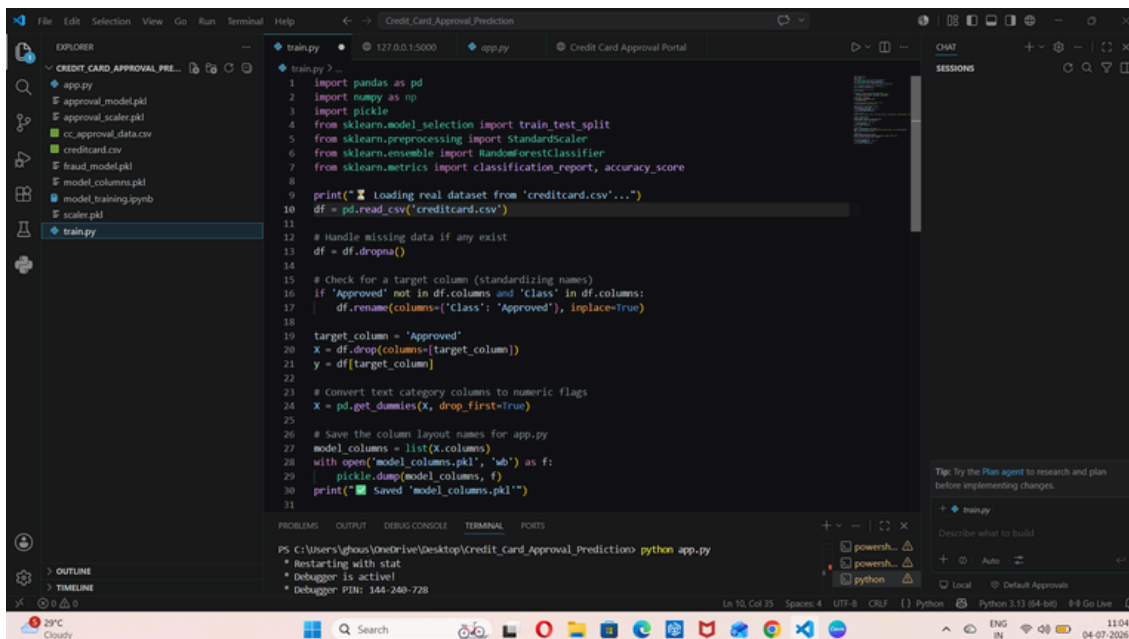
Researching classifier matrices and isolating high-accuracy model weight structures matching the financial criteria parameters.



Milestone 2: Core Functionalities Development

Establishing cleaning transformations ,missing data imputation, and exporting trained pipeline weights into portable binary formats.

Instructions: Paste a screenshot of your active model workspace training routines or code execution cells.



```

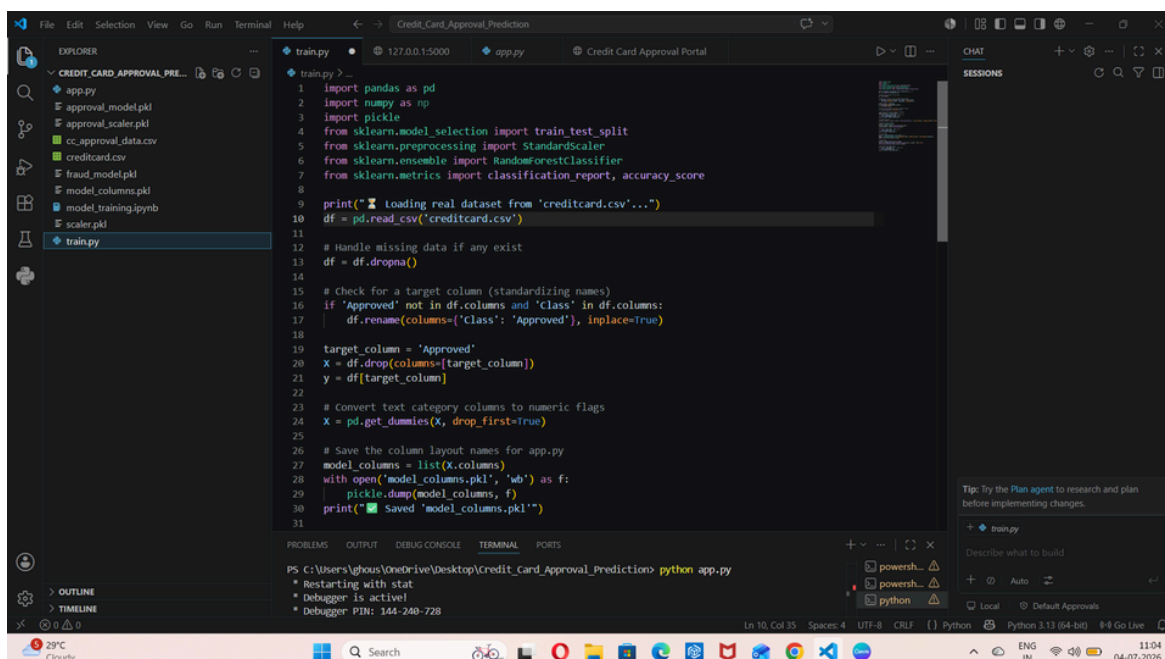
1 import pandas as pd
2 import numpy as np
3 import pickle
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import classification_report, accuracy_score
8
9 print("Loading real dataset from 'creditcard.csv'...")
10 df = pd.read_csv('creditcard.csv')
11
12 # Handle missing data if any exist
13 df = df.dropna()
14
15 # Check for a target column (standardizing names)
16 if 'Approved' not in df.columns and 'Class' in df.columns:
17     df.rename(columns={'Class': 'Approved'}, inplace=True)
18
19 target_column = 'Approved'
20 X = df.drop(columns=[target_column])
21 y = df[target_column]
22
23 # Convert text category columns to numeric flags
24 X = pd.get_dummies(X, drop_first=True)
25
26 # Save the column layout names for app.py
27 model_columns = list(X.columns)
28 with open('model_columns.pkl', 'wb') as f:
29     pickle.dump(model_columns, f)
30 print("Saved 'model_columns.pkl'")
31

```

PS C:\Users\ghous\OneDrive\Desktop\Credit_Card_Approval_Prediction> python app.py

- * Restarting with stat
- * Debugger is active!
- * Debugger PIN: 144-240-728

Milestone 3: App.py Development



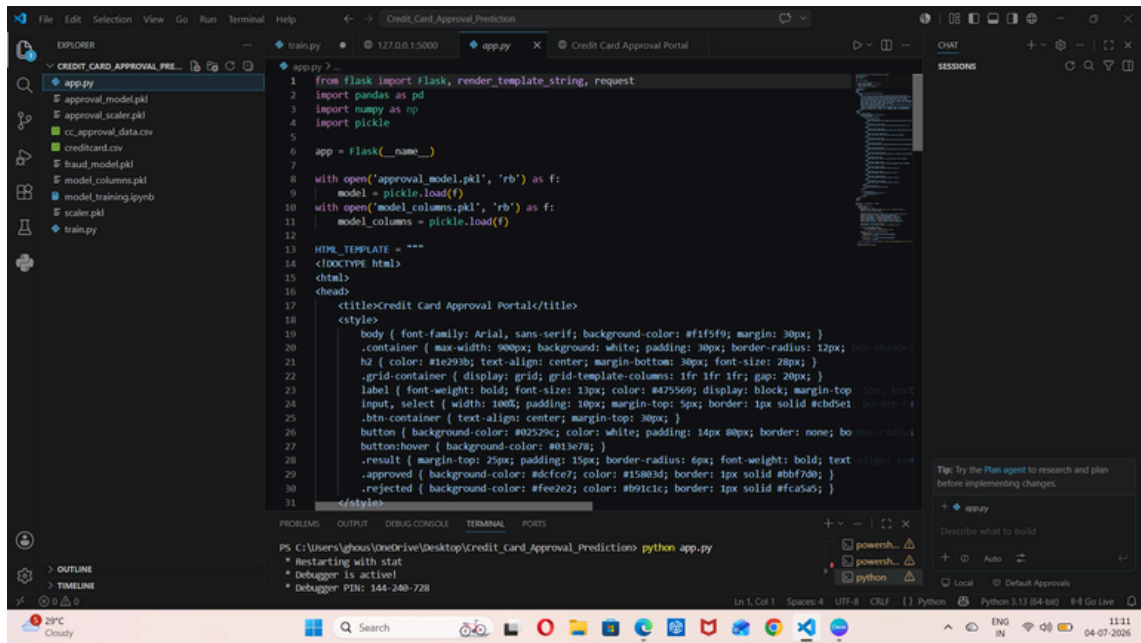
```

1 import pandas as pd
2 import numpy as np
3 import pickle
4 from sklearn.model_selection import train_test_split
5 from sklearn.preprocessing import StandardScaler
6 from sklearn.ensemble import RandomForestClassifier
7 from sklearn.metrics import classification_report, accuracy_score
8
9 print("Loading real dataset from 'creditcard.csv'...")
10 df = pd.read_csv('creditcard.csv')
11
12 # Handle missing data if any exist
13 df = df.dropna()
14
15 # Check for a target column (standardizing names)
16 if 'Approved' not in df.columns and 'Class' in df.columns:
17     df.rename(columns={'Class': 'Approved'}, inplace=True)
18
19 target_column = 'Approved'
20 X = df.drop(columns=[target_column])
21 y = df[target_column]
22
23 # Convert text category columns to numeric flags
24 X = pd.get_dummies(X, drop_first=True)
25
26 # Save the column layout names for app.py
27 model_columns = list(X.columns)
28 with open('model_columns.pkl', 'wb') as f:
29     pickle.dump(model_columns, f)
30 print("Saved 'model_columns.pkl'")
31

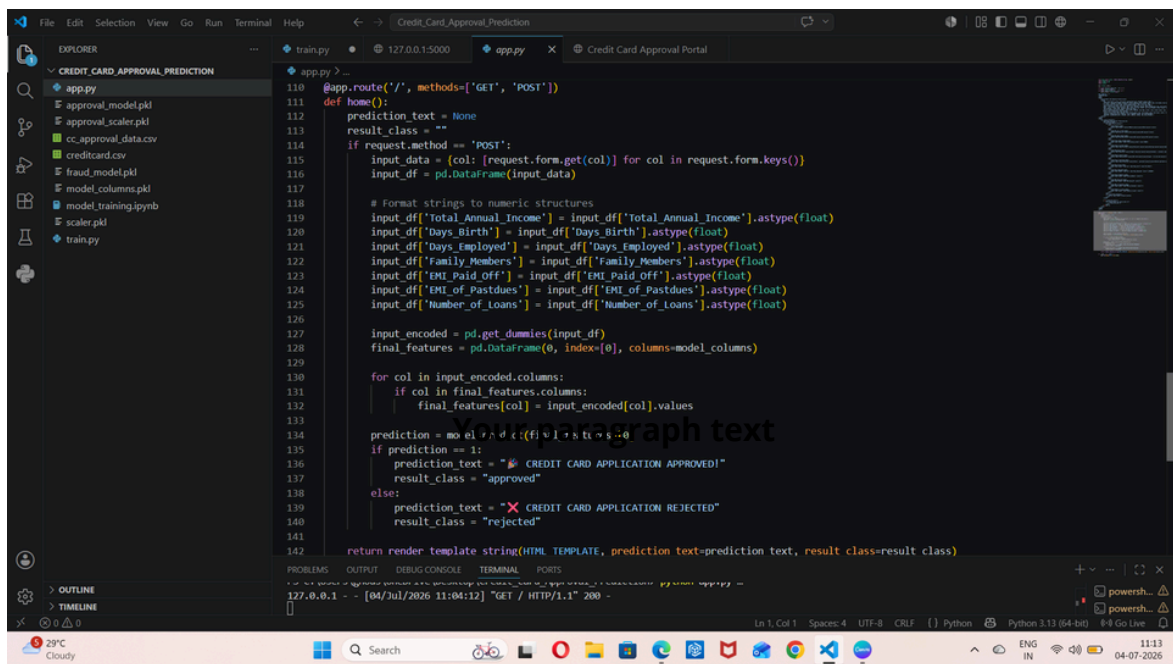
```

PS C:\Users\ghous\OneDrive\Desktop\Credit_Card_Approval_Prediction> python app.py

- * Restarting with stat
- * Debugger is active!
- * Debugger PIN: 144-240-728



app development source code



Terminal output and local hosting

```

PS C:\Users\ghous\OneDrive\Desktop\Credit_Card_Approval_Prediction> python app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 144-240-728
127.0.0.1 - - [04/Jul/2026 11:04:12] "GET / HTTP/1.1" 200 -

```

Milestone 4: Frontend Development

The screenshot shows a web browser window with the URL `127.0.0.1:5000`. The page title is "Credit Card Approval Prediction". The form contains the following fields and values:

Field	Value
GENDER	FEMALE
OWN CAR OR NOT	YES
OWN REAL STATE	NO
TOTAL ANNUAL INCOME	50000
TYPE OF INCOME	Pensioner
EDUCATION	Higher education
FAMILY STATUS	Married
TYPE OF HOUSING	House / apartment
DAYS BIRTH	29.334247
DAYS EMPLOYED	-2.095890
FAMILY MEMBERS	2
EMI PAID OFF	4
EMI OF PASTDUES	1
NUMBER OF LOANS	2

A blue "Predict" button is located at the bottom of the form.

Milestone 5: Deployment & Verification

The screenshot shows the same web browser window as in Milestone 4, but now with a green banner at the bottom of the form that reads "CREDIT CARD APPLICATION APPROVED!". The form fields and values are the same as in Milestone 4.

The background shows a code editor with the following files:

- app.py
- approval_model.pkl
- approval_scaler.pkl
- cc_approval_data.csv
- create_approval_data.py
- creditcard.csv
- fraud_model.pkl
- model_columns.pkl
- model_training.ipynb
- scaler.pkl
- train.py

The right sidebar shows a chat window with the following messages:

- Tip: Try the Plan agent to research and plan before implementing changes.
- + Credit Card Approval Portal
- Describe what to build
- + Auto
- Local Default Approvals

The bottom status bar shows the system clock as 22:03 on 03-07-2026, with a temperature of 28°C and a weather forecast of "Mostly cloudy".

Credit Card Approval Portal

127.0.0.1:5000

Summarize

Credit Card Approval Prediction

GENDER FEMALE	OWN CAR OR NOT YES	OWN REAL STATE NO
TOTAL ANUAL INCOME 50000	TYPE OF INCOME Pensioner	EDUCATION Higher education
FAMILY STATUS Married	TYPE OF HOUSING House / apartment	DAYS BIRTH 29.334247
DAYS EMPLOYED -2.095890	FAMILY MEMBERS 2	EMI PAID OFF 4
EMI OF PASTDUES 5	NUMBER OF LOANS 2	

Predict

✗ CREDIT CARD APPLICATION REJECTED

28°C Mostly cloudy

Search

ENG IN

21:47 03-07-2026